

Local Search for Job Shop under Interval Uncertainty

Hernán Díaz¹, Juan José Palacios¹, Inés González-Rodríguez^b, Irene Díaz¹, Camino R. Vela^{1,*}

^a*Department of Computing, University of Oviedo, Campus de Gijón, 33204, Gijón, (Spain), Tel: +34 985182479*

^b*Dept. of Mathematics, Statistics and Computing, University of Cantabria, Av. Los Castros s/n, 39011, Santander (Spain), Tel: +34 942202201*

Abstract

1. Introduction

1.1. Uncertain Processing Times and Due Dates

(Camino:) Esto es literal del paper postHAIS As argued in Section ??, in real life applications is common to encounter uncertainty regarding the exact time it will take to process a task on the final due date for a job. If only an upper and a lower bound of each duration and due date are known, uncertainty can be represented as a closed interval $\mathbf{a} = [a^-, a^+] = \{x \in \mathbb{R} : a^- \leq x \leq a^+\}$, with a^- and a^+ the corresponding lower and upper bounds.

Let $\mathbb{IR}$ denote the set of closed intervals. The job shop problem with total tardiness minimisation requires three arithmetic operations on $\mathbb{IR}$: addition, subtraction and maximum. These are defined by extending the corresponding operations on real numbers [1], so given two intervals $\mathbf{a} = [a^-, a^+]$, $\mathbf{b} = [b^-, b^+] \in \mathbb{IR}$, the addition is expressed as $[a^- + b^-, a^+ + b^+]$, the subtraction as $[a^- - b^+, a^+ - b^-]$ and the maximum as $[\max(a^-, b^-), \max(a^+, b^+)]$.

Comparisons are a key point in scheduling, as the “best” schedule should be the one with “minimal” total tardiness (an interval). However, there is no natural total order in the set of intervals, so an interval ranking method needs to be considered among those proposed in the literature [2]. Among the multiple

*Corresponding author

Email addresses: diazhernan@uniovi.es (Hernán Díaz), palaciosjuan@uniovi.es (Juan José Palacios), gonzalezri@unican.es (Inés González-Rodríguez), sirene@uniovi.es (Irene Díaz), crvela@uniovi.es (Camino R. Vela)

existing rankings in $\mathbb{IR}$, here we consider the following:

$$\mathbf{a} \leq_{Lex1} \mathbf{b} \Leftrightarrow a^- < b^- \vee (a^- = b^- \wedge a^+ \leq b^+), \quad (1)$$

$$\mathbf{a} \leq_{Lex2} \mathbf{b} \Leftrightarrow a^+ < b^+ \vee (a^+ = b^+ \wedge a^- \leq b^-), \quad (2)$$

$$\mathbf{a} \leq_{YX} \mathbf{b} \Leftrightarrow a^- + a^+ < b^- + b^+ \vee (a^- + a^+ = b^- + b^+ \wedge a^+ - a^- \leq b^+ - b^-), \quad (3)$$

$$\mathbf{a} \leq_{MP} \mathbf{b} \Leftrightarrow m(\mathbf{a}) \leq m(\mathbf{b}) \text{ with } m(\mathbf{a}) = (a^- + a^+)/2. \quad (4)$$

It can be seen that (1), (2) and (3) actually define total order relations in $\mathbb{IR}$ [3]. Both (1) and (2) are derived from a lexicographical order of interval extreme points. The ranking in expression (3) is proposed in [4], and the last one (mid-point order) is a particular case of the classical Hurwitz criterion and is equivalent to the one used in [5] for interval JSP.

(Camino:) Esto es nuevo. Se necesita para las demostraciones posteriores. If $a^- \leq b^-$ and $a^+ \leq b^+$ then $\mathbf{a} \leq_R \mathbf{b}$ for all previously defined rankings.

Pongo las demostraciones, aunque si son triviales se quitan (Revision: la de MP es trivial y el resto supongo que no se necesitan)

(Camino:) EN lo siguiente no he cambiado a nuestra notacion de intervalos porque estoy reproduciendo literalmente del paper de Bustince para que decidamos que poner y como From [3]: For each interval $[a, b] \subset \mathbb{R}$ we can assign a niquely point $(a, b) \in \mathbb{R}^2$ and that intervals can be ordered by means of orders of points in $\mathbb{R}^2$. The usual partial order in $\mathbb{R}^2$, given by $(a, b) \leq_2 (c, d) \iff a \leq c \wedge b \leq d$ induces the partial order of intervals which will be denoted by $\leq_2$,

$$[a, b] \leq_2 [c, d] \iff a \leq c \wedge b \leq d$$

. For the set of closed intervals in $[0, 1]$, $L([0, 1])$, given a poset $(L([0, 1]), \preceq)$, the relation between intervals $\preceq$ is an order admissible is is a linear order on $L([0, 1])$ and refines the order $\leq_2$. That means that if $a^- \leq b^-$ and $a^+ \leq b^+$ then $\mathbf{a} \preceq \mathbf{b}$ for all intervals in $[0, 1]$. The rankings $\leq_{Lex1}$, $\leq_{Lex2}$, $\leq_{YX}$ and, obviously, the order $\leq_2$ ($[a, b] \leq_2 [c, d]$ iff $a \leq c$ and $b \leq d$).

(Camino:) Supongo que es natural generalizar el concepto de orden admisible para intervalos acotados que no esten en $[0, 1]$. De hecho, supongo que estará en algún paper, pero no lo encuentro. Si es así las propiedades que necesitamos para analizar la vecindad son ciertas para todos los ordenes admisibles, entre los que se cuentan 3 de los 4 rankings definidos, y tambien para el MP, que no es un orden, pero si cumple la propiedad que buscamos, es decir, que extiende el orden $\leq_2$, es decir que

$$[a, b] \preceq [c, d] \text{ whenever } [a, b] \leq_2 [c, d]$$

(Camino:) Lo siguiente, por tanto, sobraria

Para Lex1: Suponemos $a^- \leq b^-$ and $a^+ \leq b^+$, como $a^- \leq b^-$,

- si $a^- < b^-$ entonces $\mathbf{a} \leq_{Lex1} \mathbf{b}$

- si $a^- = b^-$, como $a^+ \leq b^+$ de nuevo $\mathbf{a} \leq_{Lex1} \mathbf{b}$

Para Lex2 la demostracion es analoga

Para YX: Suponemos $a^- \leq b^-$ and $a^+ \leq b^+$, $a^- + a^+ \leq b^- + b^+$,

- si $a^- + a^+ < b^- + b^+$ entonces $\mathbf{a} \leq_{YX} \mathbf{b}$
- si $a^- + a^+ = b^- + b^+$, suponemos (RA) que $a^+ - a^- > b^+ - b^-$. Usando la igualdad y sustituyendo a^- en la desigualdad $a^+ - (b^+ + b^- - a^+) > b^+ - b^-$ y, en consecuencia, $2a^+ > 2b^+$ que es imposible con la hipótesis $a^+ \leq b^+$; de nuevo $\mathbf{a} \leq_{YX} \mathbf{b}$

Para MP es trivial: si $a^- \leq b^-$ and $a^+ \leq b^+$, $(a^- + a^+) \leq (b^- + b^+)$ luego, $\mathbf{a} \leq_{MP} \mathbf{b}$.

2. Criticality

En deterministic scheduling ES_i = longitud del camino más largo de 0 a i (*earliest starting time of i*) LS_i = longitud del camino más largo – longitud de camino más largo de $i+1$ al node *end* (*latest starting time*) $F_i = LS_i - ES_i$ (*float of i*)

Standard longest path computations in acyclic graph allow to compute all these values in $O(|E|)$. All tasks of critical path have null float.

When problem parameters are ill-known, a common way to modelling uncertainty is to define each task processing time as an interval $\mathbf{p}_i = [p_i^-, p_i^+] = \{x \in \mathbb{R} : p_i^- \leq x \leq p_i^+\}$, with p_i^- and p_i^+ the corresponding minimum and maximum values for this processing time.

Temporal analysis focuses on the computation of the minimum and maximum values of the earliest start times, latest start time and total floats.

Definition 1. A configuration ω is a tuple $\omega = (p_1, \dots, p_o)$ of durations such that $\forall i \in O, p_i \in \mathbf{p}_i$.

Ω denotes the set of all configurations. The processing time of task i in configuration ω is denoted $p_i(\omega)$. A configuration defines an instance of the corresponding deterministic scheduling problem.

Using configurations, the possible values of ES_i , LS_i for the earliest and latest starting times and F_i for float are the closed intervals defined by the following bounds:

- Best-case (minimum, lower bound) earliest starting time $ES_i^- = \min_{\omega \in \Omega} ES_i(\omega)$.
- Worst-case (maximum, upper bound) earliest starting time $ES_i^+ = \max_{\omega \in \Omega} ES_i(\omega)$.
- Worst-case (minimum, lower bound) latest starting time $LS_i^- = \min_{\omega \in \Omega} LS_i(\omega)$.
- Best-case (maximum, upper bound) latest starting time $LS_i^+ = \max_{\omega \in \Omega} LS_i(\omega)$.
- Worst-case (minimum, lower bound) float $F_i^- = \min_{\omega \in \Omega} F_i(\omega)$.

- Best-case (maximum, upper bound) float $F_i^+ = \max_{\omega \in \Omega} F_i(\omega)$.

These values are of interest for providing valuable information to the decision-maker about the level of criticality of the tasks despite the uncertain nature of processing times.

Following [6], a task i is *possibly critical* if there exists a configuration $\omega \in \Omega$ in which i is critical in the usual sense, which is equivalent to that the lower bound of its float is null, that is, $F_i^- = 0$. Special attention should be paid for *possibly critical* tasks for risk-adverse decision policies. (Camino:) Hay que definir arco posiblemente critico y camino posiblemente critico

A task i is *necessarily critical* if is critical in the usual sense in all configurations $\omega \in \Omega$, which is equivalent ([6]) to that the upper bound of its float is null, that is, $F_i^+ = 0$. A *necessarily critical* task should be carefully monitored independently of the scenario.

Is a task *necessarily critical* then $ES_i = LS_i$. However, if $ES_i = LS_i$, the task is only *possibly critical*.

Several algorithms to compute critical tasks use the notion of *extreme scenario* induce by a task subset. Let $\omega^+(Q)$, with $Q \subseteq V$, the extreme scenario such that each task $i \in Q$ is set to p_i^+ while each activity of $i \in V \setminus Q$ is set to p_i^- . Notice that the earliest starting times of each task $i \in V$ are attained on extreme scenarios induced by the empty set and V respectively:

$$ES_i^- = ES_i(\omega^+(\emptyset))$$

and

$$ES_i^+ = ES_i(\omega^+(V)).$$

Furthermore, for each task $i \in V$ it holds that the latest starting times and floats are attained for extreme scenarios [7] (yo lo he visto en [8]):

$$LS_i^+ = \max_{Q \subseteq V} LS_i(\omega^+(Q))$$

$$LS_i^- = \min_{Q \subseteq V} LS_i(\omega^+(Q))$$

$$F_i^+ = \max_{Q \subseteq V} F_i(\omega^+(Q))$$

$$F_i^- = \min_{Q \subseteq V} F_i(\omega^+(Q))$$

In fact, subsets Q can be limited to subsets of tasks located on paths from node *start* to node *end* [6].

To compute possibly and necessarily critical paths, a purely enumerative algorithm would enumerate all 2^o (where o is the number of operations) extreme scenarios and, for each of them, compute the required paths. In the particular case of minimum float, it remains a NP-hard problem [8] and the most efficient algorithm to compute it is a branch-and-bound. This is not useful in the framework of local search algorithms where the critical paths have to be computed a huge number of times.

In order to simplify the critical path computations and to define a neighbourhood structure that holds some of the highly desirable properties of neighbourhoods, we will adapt the idea proposed in [9] for Fuzzy Job Shop to the interval case. Thus, given a *feasible processing order* $\sigma \in \Sigma$ and its corresponding solution graph $G(\sigma) = (V, A \cup D(\sigma))$, we denote $G^-(\sigma)$ and $G^+(\sigma)$ the solution graphs which are identical to $G(\sigma)$ except for the cost of each arc (x, y) that is p_x^- and p_x^+ respectively. These are the directed acyclic graphs defined by $D(\sigma)$ in the disjunctive graphs representing the extreme scenarios induced by the empty set and V respectively. In coherency, we will named extreme solution graphs.

Definition 2. *A path P in $G(\sigma)$ is extreme critical if and only if P is critical in $G^-(\sigma)$ or in $G^+(\sigma)$. Nodes, arcs and blocks in a extreme critical path are critical.*

Notice that, all tasks on critical paths in $G^-(\sigma)$ and $G^+(\sigma)$ are possibly critical and contain the set of necessarily critical tasks.

Proof If i is critical in $G^-(\sigma)$ or in $G^+(\sigma)$, then $F_i(\omega^+(\emptyset)) = 0$ or $F_i(\omega^+(V)) = 0$. Thus, $F_i^- = \min_{Q \subseteq V} F_i(\omega^+(Q)) \leq \min\{F_i(\omega^+(\emptyset)), F_i(\omega^+(V))\} = 0$, so i is possibly critical.

And, if i is necessarily critical, $F_i^+ = 0$, then $F_i^+ = \max_{Q \subseteq V} F_i(\omega^+(Q)) \geq \max\{F_i(\omega^+(\emptyset)), F_i(\omega^+(V))\}$ and then $F_i(\omega^+(\emptyset)) = 0 = F_i(\omega^+(V))$, and therefore, i is critical both in $G^-(\sigma)$ and $G^+(\sigma)$. **endproof**

Given a feasible processing order σ and the schedule $\mathbf{s}(\sigma)$ built over σ (supongo que podemos poner, en otro sitio, la definicion de una planificacion que respete el orden de procesamiento y que asigne como tiempo de inicio de cada tarea el maximo entre el tiempo de fin de su predecesora en el trabajo y la predecesora en la maquina definida por σ), the makespan $[C_{max}^-, C_{max}^+]$ of the schedule is not necessarily the cost of a extreme critical path in $G(\sigma)$, but C_{max}^- and C_{max}^+ are the cost of a critical path in $G^-(\sigma)$ and in $G^+(\sigma)$ respectively.

(Camino:) cuando se defina el problema hay que dejar claro que es un *solucion optima* Since we may build a feasible schedule from a feasible task processing order, we restate the goal of the job shop problem as that of finding an optimal task processing order, in the sense that makespan for the derived schedule is optimal. However, it is not trivial to optimise a interval makespan, since neither the maximum nor its approximation define a total ordering in the set of intervals. This problem can be tackled using ranking methods. An optimal task processing order for a ranking method R is then defined as follows:

Definition 3. *Let Σ be the set of feasible task processing orders. A task processing order $\sigma_0 \in \Sigma$ is said to be optimal with respect to R (*optimal_R*) if and only if $\mathbf{C}_{\max}(\sigma_0) = \min_R \{\mathbf{C}_{\max}(\sigma) : \sigma \in \Sigma\}$*

(Camino:) Es obvio lo que es el minimo inducido por un ranking?

3. Neighbourhood

We start extending to the interval job shop, the neighbourhood structure defined in [10] for the JSP and extended for fuzzy JSP in [9].

Definition 4. Let $G(\sigma) = (V, A \cup D(\sigma))$ be a solution graph and let $\sigma_{(v)}$, with $v \in D(\sigma)$, be the processing order built from σ after reverse the processing order of tasks in arc v . The neighbourhood of the solution represented by σ is defined by

$$H(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is extreme critical}\} \quad (5)$$

If we denote with

$$H'(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is possibly critical}\}$$

it is obvious that $\forall \sigma \in \Sigma, H(\sigma) \subset H'(\sigma)$ and it is a proper subset ((Camino:) PONER UN EJEMPLO?). This reduction in the size of the set of neighbours, which is an interesting property of the neighbourhood, also allows to compute this set more efficiently, which makes it available for local search algorithms and we will see that does not affect the potential quality of solutions found by the local search whatever the ranking used to compare intervals, between the proposed rankings, that is, never improving neighbours can be lost.

For a solution graph $G(\sigma)$ and a task x , let $P\nu_x$ and $S\nu_x$ denote the predecessor and successor nodes of x on the machine sequence (predecessor and successor in $D(\sigma)$), and let PJ_x and SJ_x denote respectively the predecessor and successor nodes of x on the job sequence (predecessor and successor in A)

Proposition 1. Let $\sigma \in \Sigma$ be a feasible processing order and let $\sigma_{(v)} \in \Sigma$ be a feasible processing order obtained by the reversal an arc v which is not extreme critical in $G(\sigma)$. Then

$$C_{max}^-(\sigma) \leq C_{max}^-(\sigma_{(v)}), \text{ and } C_{max}^+(\sigma) \leq C_{max}^+(\sigma_{(v)}) \quad (6)$$

and therefore, for every ranking R (de los considerados aqui)

$$\mathbf{C}_{\max}(\sigma) \leq_R \mathbf{C}_{\max}(\sigma_{(v)}) \quad (7)$$

Proof Let $\sigma_{(v)}$ be obtained from σ by the reversal of an arc $v = (x, y)$ where v is not a extreme critical arc, that is, it is not a critical arc in $G(\sigma)$. Reversal of $v = (x, y)$ changes at most three arcs in $G(\sigma)$: (u, v) that changes to (y, x) , $(P\nu_x, x)$ (if there exists $P\nu_x$) that changes to $(P\nu_x, y)$, and $(y, S\nu_y)$ (if there exists $S\nu_y$) that changes to $(x, P\nu_y)$. Clearly, the critical arcs of $G(\sigma)$ remain unchanged in $G(\sigma_{(v)})$ after the move except, at most, $(P\nu_x, x)$ and/or $(y, S\nu_y)$. If, for example, $(P\nu_x, x)$ was critical, after reversal, this arc is replaced by $(P\nu_x, y)$ followed by (y, x) (analogously for $(y, S\nu_y)$). So, in $G^-(\sigma)$ and $G^+(\sigma)$ either there all critical arcs remains unchanged or, at most one or two are replaced by a longer path and therefore $C_{max}^-(\sigma) \leq C_{max}^-(\sigma_{(v)})$ and $C_{max}^+(\sigma) \leq C_{max}^+(\sigma_{(v)})$. $\square$

In particular, neighbours in H' which do not belong to H can never improve the makespan regardless the ranking used to compare intervals (otra vez esto solo para los rankings considerados):

Corollary 1. $\forall \sigma \in \Sigma, \forall \tau \in (H'(\sigma) - H(\sigma)) \cap \Sigma, \mathbf{C}_{\max}(\sigma) \leq_R \mathbf{C}_{\max}(\tau)$

Just as it happened in fuzzy JSP and unlike the crisp csse, the makespan of the schedule may not be the cost of a critical path. However, in each bound of the interval that represents the makespan, $C_{max}^*(\sigma), * \in \{-, +\}$ is the cost of a critical path in the corresponding solution extreme graph $G^*(\sigma)$.

For a solution graph $G(\sigma)$ and a task x , let $\mathbf{s}_x$ and $\mathbf{c}_x$ denote respectively the starting and completion times of x . We now extend some well-known notions for the crisp job shop and define the *head* $\mathbf{r}_x$ of task x as the starting time of x i.e. $\mathbf{s}_x$, in our context an interval, $\mathbf{r}_x = [r_x^-, r_x^+]$, given by:

$$\mathbf{r}_x = \max\{\mathbf{r}_{PJ_x} + \mathbf{p}_{PJ_x}, \mathbf{r}_{P\nu_x} + \mathbf{p}_{P\nu_x}\}, \quad (8)$$

and the *tail* $\mathbf{q}_x$ of task x as:

$$\mathbf{q}_x = \max\{\mathbf{q}_{SJ_x} + \mathbf{p}_{SJ_x}, \mathbf{q}_{S\nu_x} + \mathbf{p}_{S\nu_x}\}, \quad (9)$$

Clearly, $\mathbf{c}_x = \mathbf{r}_x + \mathbf{p}_x$. For each extreme graph $G^*(\sigma), * \in \{-, +\}$, r_x^* is the length of the longest path from node *start* to node x and $q_x^* + p_x^*$ is the length of the longest path from node x to node *end*. Hence, $r_x^* + p_x^* + q_x^*$ is the length of the longest path from node *start* to node *end* through node x in $G^*(\sigma)$; it is a lower bound of the corresponding extreme of the interval of the makespan, being equal to $C_{max}^*(\sigma)$ if node x belongs to a critical path in $G^*(\sigma)$. For two intervals $\mathbf{a}$ and $\mathbf{b}$ let $\mathbf{a} \simeq \mathbf{b}$ denote that one of the bounds of the intervals are equal: $a^- = b^- \vee a^+ = b^+$. The next proposition states some properties of critical arcs and tasks which follow trivially from the above definition:

Proposition 2. *If an arc (x, y) is critical, then $\mathbf{r}_x + \mathbf{p}_x \simeq \mathbf{r}_y$. A task x is critical if and only if $\mathbf{r}_x + \mathbf{p}_x + \mathbf{q}_x \simeq \mathbf{C}_{max}$.*

We may conclude that using H instead of H' in the local search procedure leads to a reduction in the number of evaluated neighbours without missing any improving neighbours, i.e. with no loss in the quality of the solution obtained by the local search. Additionally to the correlation property (a neighboring solution has a very small difference w.r.t. the current solution), which looks for a thorough exploration of the search space, we have seen that moves have a good chance to obtain an improved value of the objective function and a quite smaller size of the set $H(\sigma)$. These are three of the desirable properties of the neighbourhood ([?]).

In the following we will prove that it has the two remaining properties: feasibility and connectivity.

Theorem 1. *Let $\sigma \in \Sigma$ be a feasible task processing order; the reversal of a critical arc $v = (x, y) \in D(\sigma)$ produces a feasible processing order, i.e., $\sigma_{(v)} \in \Sigma$. In consequence, $H(\sigma) \subset \Sigma$.*

(Camino:) Proof: no hay el entorno proof en esta plantilla, si tenemos que ponerlo lo haremos If $v = (x, y)$ is critical, by definition,

$$r_y^- = r_x^- + p_x^- \vee r_y^+ = r_x^+ + p_x^+$$

Suppose by contradiction that $G(\sigma_{(v)})$ has a cycle. Since $G(\sigma)$ has no cycles and the only change in $\sigma_{(v)}$ w.r.t. σ has been the processing order between x and y , having a cycle in $G(\sigma_{(v)})$ means that there must exist an alternative path from x to y in $G(\sigma)$. From the definition of the head r_y and arithmetic of intervals, for each bound of the intervals, $*$ $\in \{-, +\}$ we have that

$$r_y^* \geq r_{PJ_y}^* + p_{PJ_y}^*$$

and together with the existence of an alternative path from x to y this means that:

$$r_y^* \geq r_{SJ_x}^* + p_{SJ_x}^* + p_{PJ_y}^* \geq r_x^* + p_x^* + p_{SJ_x}^* + p_{PJ_y}^*$$

But being critical, $r_y^- = r_x^- + p_x^- \vee r_y^+ = r_x^+ + p_x^+$, and for the bound $*$ ($-$ or $+$) we have:

$$r_x^* + p_x^* \geq r_x^* + p_x^* + p_{SJ_x}^* + p_{PJ_y}^*$$

which is absurd, because all task durations are strictly positive. Therefore, there can be no cycles in $G(\sigma_{(v)})$, that is, $\sigma_{(v)}$ is a feasible task processing order. $\square$ (Camino:) Aquí termina la prueba larga

Sketch of Proof (Camino:) puede que valga con algo como esto Suppose by contradiction that $G(\sigma_{(v)})$ has a cycle; this means that there exists an alternative path from x to y in $G(\sigma)$. This, together with (x, y) being critical, leads to the inequality $r_x^* + p_x^* \geq r_x^* + p_x^* + p_{SJ_x}^* + p_{PJ_y}^*$ for some bound of the intervals $*$ $= -, +$, which contradicts the fact that all task durations are strictly positive. $\square$

Notice that feasibility means that local search is automatically limited to the subspace of feasible task orders. It has the additional advantage of avoiding feasibility checks for the neighbours, hence increasing the efficiency of the local search procedure (reducing computational load) and avoiding the loss of feasible solutions that is usually encountered for feasibility checking procedures (cf. [?]).

(Camino:) Voy a meter un resultado que seguro que ya existe en la literatura. Supongo que lo lógico sera ponerlo como comentario en algun sitio, en lugar de como proposicion. De momento lo dejo demostrado, pero no creo que deba ir aqui porque es un resultado para crisp

Proposition 3. *For the JSP crisp, given $G(\sigma) = (V, A \cup D(\sigma))$ and $\overline{G(\sigma)} = (V, A \cup \overline{D(\sigma)})$ where $\overline{D(\sigma)}$ denotes the transitive closure of $D(\sigma)$. The length of the critical paths in both graphs is the same.*

(Camino:) proof Let P a critical path in G and Q a critical path of $\overline{G}$. As the set of arcs of G is contained in the set of arcs of $\overline{G}$, P is a path in $\overline{G}$, then $\|P\| \leq \|Q\|$. To complete the proof, if all arcs of Q were in G , then $\|Q\| \leq \|P\|$.

Otherwise, suppose (R.A.) that there exists some arc in Q that doesn't belongs to G . For each one of these arcs (x, y) , there exist in G a path from x to y with more than one arc (longer, because the weight of the first arc is also p_x). These paths are also paths in $\overline{G}$. Replacing in Q each of those arcs for the corresponding paths a new path, Q' , longer than Q , exists in $\overline{G}$, which is absurd.

Proposition 4. *Let $\sigma \in \Sigma$ be a feasible task processing order, $G(\sigma) = (V, A \cup D(\sigma))$ its disjunctive graph and σ_0 an optimal processing order, regardless of the chosen ranking. Let*

$$V_\sigma(\sigma_0) = \{v = (x, y) \in D(\sigma) : v \text{ is extreme critical}, (y, x) \in \overline{D(\sigma_0)}\} \quad (10)$$

where $\overline{D(\sigma)}$ denotes the transitive closure of $D(\sigma)$; i.e., $V_\sigma(\sigma_0)$ is the set of critical arcs (x, y) in $D(\sigma)$ such that there exists a path from y to x in $E(\sigma_0)$. If holds that, if σ is not optimal, $V_\sigma(\sigma_0) \neq \emptyset$ or, equivalently, if $V_\sigma(\sigma_0) = \emptyset$ then σ is optimal.

(Camino:) **Proof** We shall first prove that, if σ is not optimal, there is at least a critical arc in $D(\sigma)$. Secondly, we shall prove that if σ is not optimal, then there exists some critical arc $v = (x, y)$ such that $(y, x) \in \overline{D(\sigma_0)}$.

If there are no critical arcs in $D(\sigma)$, that means that all critical arcs in $G(\sigma)$ belong to A . (Camino:) Juanjo sugiere una demo alternativa de este parrafo que puede sea mas sencilla: As A are also arcs of $G(\sigma_0)$, all critical paths in $G(\sigma)$ are paths in $G(\sigma_0)$ and, as σ_0 is optimal, these path are critical in $G(\sigma_0)$, and in consequence σ is optimal and this proof serves for every ranking method. Therefore, all critical paths in $G^-(\sigma)$ and in $G^+(\sigma)$ belong to A . Hence, in $G^-(\sigma)$ (and also in $G^+(\sigma)$) there exists a critical path where all arcs belong to A and such path is optimal in $G^-(\sigma)$ (and $G^+(\sigma)$) (for every bound $* \in \{-, +\}$, a path where all arcs belong to the same job is a lower bound of C_{max}^*). Therefore, σ is optimal regardless the ranking method because each bound of $\mathbf{C}_{\max}$ is minimum.

Let us now asume that all critical arcs $v = (x, y) \in D(\sigma)$ verify that $(x, y) \in \overline{D(\sigma_0)}$. The set of critical arcs in $G(\sigma)$ is the union of the set of critical arcs across the extreme graphs. Therefore, the assumption means that all critical arcs in $D^-(\sigma)$ belong to the transitive closure $\overline{D^-(\sigma_0)}$ (and analogously for $D^+(\sigma)$). Hence, a critical path P^- in $G^-(\sigma)$ is a path in $\overline{G^-(\sigma_0)} = (V, A \cup \overline{D^-(\sigma_0)})$ (and a critical path P^+ in $G^+(\sigma)$ is also a path in $\overline{G^+(\sigma_0)} = (V, A \cup \overline{D^+(\sigma_0)})$). Let Q^- denote an arbitrary critical path in $\overline{G^-(\sigma_0)}$ and let $\|Q^-\|$ denote its length. Since P^- is a path in $\overline{G^-(\sigma_0)}$, and as the critical path in $G^-(\sigma_0)$ has the same length that the critical path in $G^-(\sigma_0)$ (that is $C_{max}^-(\sigma_0)$), it holds that:

$$\begin{aligned} C_{max}^-(\sigma) &= \|P^-\| \leq \|Q^-\| = C_{max}^-(\sigma_0) \\ C_{max}^+(\sigma) &= \|P^+\| \leq \|Q^+\| = C_{max}^+(\sigma_0) \end{aligned}$$

That is, $\mathbf{C}_{\max}(\sigma) \leq_R \mathbf{C}_{\max}(\sigma_0)$ for all ranking R .

But, for each ranking R since σ_0 is *optimal* _{R} , $\mathbf{C}_{\max}(\sigma_0) \leq_R \mathbf{C}_{\max}(\sigma)$. Thus, for all rankings admissible, as induces lineal orders, we have that $\mathbf{C}_{\max}(\sigma_0) = \mathbf{C}_{\max}(\sigma)$ and, hence, σ is optimal. In the case of the ranking MP , it holds that

$$(\|Q^-\| + \|Q^+\|) \leq (\|P^-\| + \|P^+\|)$$

Both inequalities mean that $MP(\mathbf{C}_{\max}(\sigma_0)) = MP(\mathbf{C}_{\max}(\sigma))$ and, hence, σ is optimal also in ranking MP . (Camino:) [Fin de la demo del lema](#)

Theorem 2. *H verifies the connectivity property, that is, for every non-optimal task processing order σ we may build a finite sequence of transitions of H leading from σ to a globally optimal processing order σ_0 .*

(Camino:) [Demo larga, con lema](#) Let σ_0 be any optimal processing order and let the sequence $\{\lambda_k\}_{k \geq 0}$ of processing orders be given by the following recursive definition:

$$\lambda_0 = \sigma$$

$$\lambda_{k+1} \text{ is obtained from } \lambda_k \text{ by reversal of an arc } v \in V_{\lambda_k}(\sigma_0)$$

Notice that the reversal of an arc in $V_{\lambda_k}(\sigma_0)$ is a move from H so, by Theorem 1, $\forall k \lambda_k \in \Sigma$ (i.e., all task processing orders in the sequence are feasible solutions). Let us prove that the above sequence is finite. For any processing order $\sigma \in \Sigma$, we define the following sets:

$$\begin{aligned} M_\sigma(\sigma_0) &= \{v = (x, y) \in D(\sigma) : (y, x) \in \overline{D(\sigma_0)}\} \\ \overline{M_\sigma(\sigma_0)} &= \{v = (x, y) \in \overline{D(\sigma)} : (y, x) \in \overline{D(\sigma_0)}\} \end{aligned}$$

Clearly, $V_\sigma(\sigma_0) \subset M_\sigma(\sigma_0)$ and $M_\sigma(\sigma_0) \subset \overline{M_\sigma(\sigma_0)}$. Let $\|M_\sigma(\sigma_0)\|$ and $\|\overline{M_\sigma(\sigma_0)}\|$ denote their cardinals. By definition of λ_k , if $\|\overline{M_{\lambda_k}(\sigma_0)}\| > 0$ then

$$\|\overline{M_{\lambda_{k+1}}(\sigma_0)}\| = \|\overline{M_{\lambda_k}(\sigma_0)}\| - 1.$$

Therefore, for $k^* = \|\overline{M_\sigma(\sigma_0)}\|$, we have that $\|\overline{M_{\lambda_{k^*}}(\sigma_0)}\| = 0$. Given that $V_\sigma(\sigma_0) \subseteq M_\sigma(\sigma_0) \subseteq \overline{M_\sigma(\sigma_0)}$, this implies that $V_{\lambda_{k^*}}(\sigma_0) = \emptyset$ so, by Proposition 4, λ_{k^*} is optimal. (Camino:) [Fin de la demo que usa el lema](#)

Sketch of proof. The first step is to prove that if $V_\sigma(\sigma_0) = \{v = (x, y) \in D(\sigma) : v \text{ is critical}, (y, x) \in \overline{D(\sigma_0)}\} = \emptyset$ then σ is *optimal* _{R} for all ranking R , where $\overline{D(\sigma_0)}$ denotes the transitive closure of $D(\sigma_0)$. To do this, first prove that if σ is not optimal, there exists at least a critical arc in $D(\sigma)$. Then prove that if σ is not optimal, there exists some critical arc $(x, y) \in D(\sigma)$ such that $(y, x) \in \overline{D(\sigma_0)}$, using the fact that the set of critical arcs in $G(\sigma)$ is the union of critical arcs in $G^-(\sigma)$ and in $G^+(\sigma)$. The second step is to define the sequence $\{\lambda_k\}_{k \geq 0}$, where $\lambda_0 = \sigma$, and λ_{k+1} is obtained from λ_k by reversal of an arc $v \in V_{\lambda_k}(\sigma_0)$. To prove that this sequence is finite, define $M_\sigma(\sigma_0) = \{v = (x, y) \in D(\sigma) : (y, x) \in \overline{D(\sigma_0)}\}$ and $\overline{M_\sigma(\sigma_0)} = \{v = (x, y) \in \overline{D(\sigma)} : (y, x) \in \overline{D(\sigma_0)}\}$,

notice that if $\|\overline{M_{\lambda_k}(\sigma_0)}\| > 0$ then $\|\overline{M_{\lambda_{k+1}}(\sigma_0)}\| = \|\overline{M_{\lambda_k}(\sigma_0)}\| - 1$ and use the property from step 1. $\square$

As mentioned above, connectivity is an important property for any neighbourhood used in local search. It ensures the non-existence of starting points from which local search cannot reach a global optimum. It also ensures asymptotic convergence in probability to a globally optimal order. Additionally, although the neighbourhood structure is used in a heuristic procedure in this paper, the connectivity property would allow to design an exact method for fuzzy job shop, like the well-known Branch and Bound for the crisp case.

Acknowledgments

All authors are supported by MEC-FEDER Grant .

References

- [1] R. E. Moore, R. B. Kearfott, M. J. Cloud, Introduction to Interval Analysis, Society for Industrial and Applied Mathematics, 2009 (2009).
- [2] S. Karmakar, A. K. Bhunia, A comparative study of different order relations of intervals, *Reliable Computing* 16 (2012) 38–72 (2012).
- [3] H. Bustince, J. Fernandez, A. Kolesárová, R. Mesiar, Generation of linear orders for intervals by means of aggregation functions, *Fuzzy Sets and Systems* 220 (2013) 69–77 (2013).
- [4] Z. Xu, R. R. Yager, Some geometric aggregation operators based on intuitionistic fuzzy sets, *International Journal of General Systems* 35 (4) (2006) 417–433 (2006).
- [5] D. Lei, Interval job shop scheduling problems, *International Journal of Advanced Manufacturing Technology* 60 (2012) 291–301 (2012). doi:10.1007/s00170-011-3600-3.
- [6] D. Dubois, H. Fargier, J. Fortin, Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks, *Journal of Intelligent Manufacturing* 16 (2005) 407–421 (2005).
- [7] D. Dubois, H. Fargier, P. Fortemps, Fuzzy scheduling: Modelling flexible constraints vs. coping with incomplete knowledge, *European Journal of Operational Research* 147 (2003) 231–252 (2003).
- [8] C. Artigues, C. Briand, T. Garaix, Temporal analysis of projects under interval uncertainty, in: C. Schwindt, J. Zimmermann (Eds.), *Handbook on Project Management and Scheduling Vol. 2*, Springer, 2015, pp. 911–927 (2015). doi:10.1007/978-3-319-05915-0-11.

- [9] I. González Rodríguez, C. R. Vela, J. Puente, R. Varela, A new local search for the job shop problem with uncertain durations, in: Proceedings of the Eighteenth International Conference on Automated Planning and Scheduling (ICAPS-2008), AAAI Press, Sidney, 2008, pp. 124–131 (2008).
- [10] P. Van Laarhoven, E. Aarts, K. Lenstra, Job shop scheduling by simulated annealing, *Operations Research* 40 (1992) 113–125 (1992).